

PRD — SIH26092: AI-Driven Scheme Matching for Marginalized Entrepreneurs

Ministry of Social Justice & Empowerment (NSFDC channel-finance scheme) · Team: [fill in] · MUJ, SIH 2026 v2 — reconciles the team's own NSFDC/DPDPA research report with direct verification against nsfdc.nic.in and official DPDPA sources.

0. Snapshot

A WhatsApp-first (web later) assistant that takes an SC beneficiary through three things in one flow: which NSFDC credit/education scheme they actually qualify for, what their EMI would look like, and which real Channel Partner (SCA/bank/NBFC-MFI) near them can process it — filtered by whether that partner is even allowed to disburse funds right now. Built by extending the existing GrantBot codebase (LangGraph + Gemini), not starting fresh. Backend-first; frontend deferred. Free/open-source stack throughout.

1. Problem statement (condensed from the official PS)

NSFDC provides concessional credit to SC beneficiaries (family income $\leq$ ₹5 lakh per the PS) through a Channel Finance System — 100+ Channel Partners across four categories: State Channelizing Agencies (SCAs), Public Sector Banks, Regional Rural Banks, and NBFC-MFIs/State Cooperative Banks. Beneficiaries can't tell which scheme fits them, can't find an authorized partner, and applications get misrouted to partners that can't actually disburse — causing delays.

Required deliverables (per the PS itself):

Smart Scheme Recommender — inputs: project type, cost, income, education status → best-fit scheme
Financial Calculator — EMI projection accounting for scheme-specific rate/limit/moratorium
Geo-Spatial Partner Locator & Router — nearest eligible partner, filtered by fund-utilization/NPA eligibility

2. Goals & non-goals

In scope now: all three deliverables, fully working on WhatsApp, multilingual (text first, voice as stretch), reliable and idempotent. **Deferred:** web dashboard polish, full national partner coverage, live prudential-norm data feeds (the *logic* is real — see §6.2 — the *data* is mocked), production-scale rollout, the broader scheme universe beyond the PS's named three (Udyam Nidhi, VCF-SC, Stand-Up India, PM-USP CSIS exist and are real, but the PS names three specific products — build those with full rigor before expanding scope).

3. Users

Primary: SC beneficiary reaching the bot over WhatsApp, possibly on a low-end device, possibly more comfortable in a regional language than English. **Secondary (context, not a build target):** Channel Partner/SCA staff drowning in misrouted applications — this is the "transparency and efficiency" half of the PS's stated impact goals.

4. Success criteria — mapped to the judging rubric

5 criteria × 20 marks = 100, averaged across all judges (not best-of).

Criterion	What earns it here
Problem Understanding	Talk in the PS's own institutional language — Channel Finance System, prudential norms, misrouted applications — not generic "helping poor entrepreneurs" framing
Innovation & Uniqueness	The prudential-norm-aware router (§6.2) — easy to miss in the PS, most teams will — plus multilingual, WhatsApp-native delivery
Technical Feasibility	Working end-to-end on real scheme data; explicit about what's mocked and why
Impact & Scalability	Lead with a concrete beneficiary persona; close on this pattern existing beyond NSFDC (NBCFDC runs a near-identical structure for OBCs)
Presentation & Execution	Live demo on WhatsApp itself; every team member can field a question on any component

5. System architecture

Reused from GrantBot: WhatsApp delivery (Twilio Sandbox) + web dashboard shell; LangGraph agent orchestration + FastAPI, Gemini-powered.

New, three-tier recommendation pipeline (this is the one real upgrade from the previous version of this doc — a pure rules engine was too rigid, a pure LLM/RAG approach was too unreliable for a numeric eligibility decision; the fix is a pipeline, not a single technique):

Tier 1 — Deterministic eligibility filter. Hard cutoffs (income, project cost, category, age) evaluated as plain boolean logic against a rules table. 100% precision on boundary cases. No API call.

Tier 2 — Light semantic matching, only for free text. When a user describes their project in their own words ("I want to open a small tailoring shop") instead of picking from buttons, a small embedding lookup (or even simple keyword matching against project-type categories — doesn't need to be heavy RAG) maps that description to a

structured project-type field Tier 1 can use. This is the *only* place fuzzy matching touches the pipeline — it feeds Tier 1, it never overrides it.

Tier 3 — Institutional/prudential routing. Covered in §6.2 below.

Data: SQLite in WAL mode for the hackathon/demo scope — sufficient for this scale and free. Write the data-access layer without SQLite-specific query patterns baked in, so a later move to PostgreSQL/PostGIS (the more "correct" choice at real production scale, per the team's own research) isn't a rewrite, just a swap.

Use LangGraph's own checkpointers for conversation state, not a hand-rolled table. This is a genuine improvement worth making: LangGraph ships a `CheckpointSaver` abstraction purpose-built for exactly "where is this user in the flow" — `from langgraph.checkpoint.sqlite import SqliteSaver` for the hackathon/demo (swap to `from langgraph.checkpoint.postgres import PostgresSaver` if concurrent load at finale calls for it), keyed by a `thread_id` per user. Verified current and stable. Using the framework's own persistence instead of a custom `conversation_state` table removes a whole class of the state-sync bugs that plausibly caused GrantBot's original issue in the first place — it's specifically designed for this exact failure mode.

The message chain, end to end (unchanged from the prior version of this doc — this is the fix for GrantBot's actual reliability failures, not a place that needed new research):

Webhook verifies the request's HMAC-SHA256 signature (Meta and Twilio both sign payloads this way), checks idempotency, enqueues, returns `200` immediately — no slow work in the handler.

Worker reads user state atomically.

Structured input (button/list/Flow) skips straight to Tier 1; free text goes through Tier 2 first.

Tier 1 evaluates eligibility — pure computation.

Generation step checks cache before calling any LLM (§7).

State written back atomically, in the same transaction it was read in.

Response sent via a separate outbound WhatsApp API call.

6. The three-tier recommendation engine, in detail

6.1 Tier 1 — Eligibility rules (reference implementation)



python

```

from pydantic import BaseModel
from typing import List

class UserProfile(BaseModel):
    category: str          # "SC" for this PS's scope
    annual_income: float
    project_cost: float
    gender: str
    age: int
    is_student: bool = False

class SchemeRule:
    def __init__(self, code, name, max_income, max_cost, min_age, max_age,
                  target_category, requires_student=False):
        self.code, self.name = code, name
        self.max_income, self.max_cost = max_income, max_cost
        self.min_age, self.max_age = min_age, max_age
        self.target_category = target_category
        self.requires_student = requires_student

    def evaluate(self, user: UserProfile) -> bool:
        if user.category != self.target_category: return False
        if user.annual_income > self.max_income: return False
        if user.project_cost > self.max_cost: return False
        if not (self.min_age <= user.age <= self.max_age): return False
        if self.requires_student and not user.is_student: return False
        return True

def evaluate_eligible_schemes(user: UserProfile, rules: List[SchemeRule]) -> List[str]:
    return [r.name for r in rules if r.evaluate(user)]

```

Every threshold below is a config value, not a hardcoded constant — deliberately, because the honest state of these numbers is that they conflict across sources, including across NSFDC's own scheme pages:

Scheme (PS's own name)	What we could confirm	Source & caveat
Micro Finance Scheme (≤₹1.40L)	Likely corresponds to NSFDC's general "Micro Credit Finance" product, distinct from the women-only Mahila Samriddhi Yojana which shares the same ₹1.40L ceiling	PS text is explicit on the ₹1.40L figure; MSY vs. general Micro Credit Finance distinction is from the team's research, not independently confirmed
Term Loan (≤₹50L)	NSFDC's own Term Loan page states a flat 4% (to SCA) / 8% (to beneficiary) structure, 7-year repayment incl. 6-month moratorium. A separate, distinctly-named NSFDC product, "Utkarsh Loan," covers ₹10-50L specifically at 9% to the beneficiary.	Directly verified on nsfdc.nic.in — but the two pages don't agree with each other's implied scope, and neither matches the PS's own "6.5-15%" framing exactly
Educational Loan Scheme	NSFDC's FAQ states 1.5%/4% (SCA/beneficiary), 0.5% women's rebate, 6-month moratorium, 7-year repayment. Third-party sources give amount ceilings anywhere from ₹10L to ₹40L depending on the site.	nsfdc.nic.in FAQ confirms the rate structure; the amount ceiling is genuinely unresolved across sources
Income eligibility	PS states ≤₹5L family income. Multiple third-party sources state ≤₹3L (rural) / ₹3.5L (urban) for NSFDC schemes generally.	Use the PS's ₹5L as your build default — it's literally what you're being judged against — but make it a one-line config change

What to actually do with this table: build against the PS's own stated numbers as the default. Keep every number in one config file. Budget 20 minutes before your internal round to have one person actually read nsfdc.nic.in's specific scheme pages (not aggregator blogs — several of those, particularly lending-marketplace sites, appear to conflate NSFDC with their own commercial products) and confirm or adjust. This table, presented honestly, is *also* a legitimate part of your Problem Understanding pitch — "even official sources disagree with each other, which is exactly the fragmentation this PS is asking us to solve" is a true, on-message thing to say out loud.

6.2 Tier 3 — Institutional/prudential routing (the differentiator)

This is the part of the PS most teams will skim past. The "fund-utilization eligibility" language in the brief maps to real, nameable NSFDC prudential norms:

SCAs: fresh fund releases require 100% cumulative utilization of prior disbursements as of the end of the preceding month, and zero overdue payments to NSFDC.

RRBs: disbursement eligibility requires Net NPA below 15% in at least 3 of the preceding 6 financial years, and a net profit in at least 3 of those 6 years.

Build the *rule* for real — it's genuinely public information and your rules engine can encode it exactly. Mock only the *live per-branch data* feeding it (today's actual utilization %, today's actual NPA figure for a given branch) — nobody outside NSFDC has that in real time, and no other team will either.



```
python
import math
from pydantic import BaseModel
from typing import List

class ChannelPartnerBranch(BaseModel):
    branch_id: str; name: str
    agency_type: str          # "SCA", "PSB", "RRB", "NBFC-MFI"
    latitude: float; longitude: float
    net_npa_percentage: float # mocked, but the threshold check is real
    cumulative_utilization: float
    has_active_overdues: bool

def haversine_km(lat1, lon1, lat2, lon2) -> float:
    R = 6371.0
    dlat, dlon = math.radians(lat2 - lat1), math.radians(lon2 - lon1)
    a = (math.sin(dlat/2)**2 +
         math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(dlon/2)**2)
    return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

def find_viable_partners(user_lat, user_lon, branches: List[ChannelPartnerBranch],
                        radius_km: float = 50.0) -> List[dict]:
    viable = []
    for b in branches:
        if b.agency_type == "SCA" and (b.cumulative_utilization < 1.0 or b.has_active_overdues):
            continue
        if b.agency_type == "RRB" and b.net_npa_percentage >= 15.0:
            continue
        d = haversine_km(user_lat, user_lon, b.latitude, b.longitude)
        if d <= radius_km:
            viable.append({"branch_id": b.branch_id, "name": b.name,
                          "agency_type": b.agency_type, "distance_km": round(d, 2)})
    return sorted(viable, key=lambda x: x["distance_km"])
```

Getting lat/long without a paid geocoding API: WhatsApp supports native one-tap location sharing — no LLM, no geocoding step, no cost. For users who type a PIN code or district name instead, pair it with a free, open PIN-code-to-coordinates dataset (India Post publishes this; several open mirrors exist) rather than a paid geocoder.

Seed data, not a demo-day scramble: real State Channelizing Agencies exist and are known institutions — e.g. Maharashtra's Mahatma Phule Backward Class Development Corporation, Tamil Nadu's Adi Dravidar Housing and Development Corporation (TAHDCO), the West Bengal SC ST Development and Finance Corporation. Use real

names like these for your seed set; only the per-branch utilization/NPA figures attached to them need to be the clearly-labeled mocked values, not the institutions themselves.

6.3 The calculator — tiered rates, real moratorium handling

The team's research surfaced a genuinely important gap in *every* existing platform (myScheme, JanSamarth, Stand-Up India portal, Vidya Lakshmi): none of them model multi-tiered interest rates or moratorium-period interest treatment correctly. That's your calculator's actual differentiation, not just "it computes an EMI."

```
python
def calculate_emi(principal: float, tenure_years: int, scheme_code: str,
                  gender: str, moratorium_months: int = 0) -> dict:
    # Rate lookup belongs in the same config table as §6.1 — shown inline for clarity
    if scheme_code == "TERM_LOAN":
        annual_rate = 6.0 if principal <= 500_000 else 8.0 if principal <= 1_000_000 else 9.0
    elif scheme_code == "EDU_LOAN":
        annual_rate = 4.0 - (0.5 if gender.upper() == "FEMALE" else 0.0)
    else:
        annual_rate = 8.0 # fallback — should never actually be hit once config is complete

    r = (annual_rate / 100) / 12
    n = tenure_years * 12
    repay_months = n - moratorium_months
    moratorium_monthly_interest = principal * r
    emi = (principal * r * (1 + r)**repay_months) / ((1 + r)**repay_months - 1) if r > 0 else principal / repay_months

    return {
        "annual_rate": annual_rate,
        "monthly_emi": round(emi, 2),
        "moratorium_monthly_interest": round(moratorium_monthly_interest, 2),
        "repayment_months": repay_months,
    }
```

Support all three moratorium treatments your research identified, since different schemes actually use different ones: simple-interest-during-moratorium (shown above), capitalized-interest (unpaid interest added to principal before recalculating EMI), and government-subsidy (beneficiary pays ₹0 during moratorium — relevant if you extend into PM-USP CSIS-style products later).

Don't train or fine-tune anything. No labeled dataset exists to justify it, and self-hosting the result reintroduces the reliability problem this whole rebuild exists to fix. The eligibility decision is Tier 1's job, not the model's — a smaller job for the model beats a smarter model.

Primary: Gemini (Flash-Lite for the Tier-2 text-to-category extraction — lighter task, higher free daily quota; Flash/Pro for generation, only on a cache miss). Google has dedicated support for Hindi and eight other Indian languages — keep Gemini as default for anything a beneficiary actually reads.

Free-tier reality (verify against your live AI Studio dashboard — these move):

Model	RPM RPD	
Gemini 2.5 Pro	5	100
Gemini 2.5 Flash	10	250
Gemini 2.5 Flash-Lite 15	1,000	

Fallback, exactly as you proposed — open-source model + translator bridge:

On a Gemini failure/429, route the (already-extracted, mostly-English-or-structured) request to an open-source model via **OpenRouter** — DeepSeek, Kimi K2, and GLM-4.5-Air are all free there, alongside Qwen/Llama/Gemma. One API key, roughly 20 RPM / 200 RPD per free model, and rotating across several multiplies your effective daily capacity well past Gemini alone. **Before wiring this in, turn on "ZDR Endpoints only" and disable "free endpoints that may train on inputs" in OpenRouter's account privacy settings** — some free endpoints train on inputs by default, and this is the confirmed, working way to exclude them. Also worth a quick check against OpenRouter's current API docs for a per-request `provider` field covering data collection preference, which would let you enforce this in code rather than relying on the dashboard setting alone — that specific field wasn't confirmable at the time of writing.

For anything that has to be generated *in* a regional language and Gemini isn't available: generate in English with the fallback model (where these models are genuinely strong, benchmark-wise), then translate with **AI4Bharat's IndicTrans2** — open-source, MIT-licensed, purpose-built for exactly this, small enough (1-1.1B params) to self-host on modest hardware or call via a free Hugging Face Space. This is a better answer than hoping an open-source generalist model's Hindi is good enough, because you're not hoping — you're using a model built specifically to be good at that one job.

AI4Bharat also publishes open Hindi/regional **ASR and TTS models** — the same family solves your voice-note stretch goal later with no new vendor relationship.

Caching is still what actually keeps you inside any of these quotas. With effectively 3 schemes, the explanation text is almost entirely reusable per (scheme, language) pair — cache it, regenerate only the personalized numbers.

8. Data model

SQLite, WAL mode. Core tables: `users`, `conversation_state` (single source of truth for flow position), `processed_messages` (idempotency), `recommendation_cache` (profile fingerprint + language → cached explanation), `schemes` and `channel_partners` (static reference data, config-driven per §6.1's table). Full DDL is in the earlier backend report if you need it verbatim.

9. WhatsApp integration

Structured input (buttons, lists, Flows) over free text wherever the answer set is fixed — this is both more reliable than LLM parsing on the load-bearing path and closer to how a real business uses WhatsApp. Sandbox is fine for the demo itself; Meta's direct Cloud API is the closer-to-production route, and since your usage is entirely reactive, it should fall inside the free 24-hour service window with no per-message cost — confirm against Meta's current pricing page before committing. One shared service layer behind both WhatsApp and the future web backend, so the two channels can't quietly disagree. Language selection via buttons at the start, not auto-detection.

10. Non-functional requirements

Reliability: idempotent message processing, atomic state read/write, async webhook handling — the direct fix for GrantBot's prior failures.

Security & privacy — this is DPDPA-governed data, treat it that way: caste/social category is high-sensitivity personal data; income and any Aadhaar/PAN-adjacent fields are critical-sensitivity. The Digital Personal Data Protection Rules were notified 13 November 2025 on a three-stage rollout, not a single grace period: the Data Protection Board and rule-making powers came into force immediately; Consent Manager registration activates 13 November 2026; the substantive provisions — notice, consent, data-principal rights, fiduciary obligations, and the penalty clauses themselves (up to ₹250 crore per instance for inadequate security safeguards) — don't take effect until 13 May 2027. That's a stronger, more precise pitch line than a vague "grace period": you're building compliance-by-design ahead of a dated deadline, not scrambling to dodge an already-active penalty regime. Practically, for a hackathon build: get explicit consent before collecting category/income data (a one-time WhatsApp message at conversation start is enough), don't log this data in plaintext beyond what's needed, encrypt what you can at rest, and have an actual deletion plan for test/demo data. Worth knowing for the pitch, but not for the build: the Act does define registered "Consent Manager" intermediaries and a "Significant Data Fiduciary" tier with extra obligations — both real, correctly-named parts of the framework — but confirmed current guidance is that routing consent through a Consent Manager is optional, not mandatory; collecting consent directly, as above, is fully compliant. Don't build more than that for a hackathon. Also: verify Twilio/Meta webhook signatures; confirm no API key has ever been committed to the public repo (rotate if so); validate all webhook input; rate-limit the webhook endpoint separately from LLM rate limiting.

Cost: \$0 required to build or demo — SQLite, Gemini free tier, OpenRouter free models, IndicTrans2 (self-hosted or free HF Space), Twilio Sandbox.

11. Build phases

Phase 0 — Before internals: webhook→queue→worker skeleton with idempotency and atomic state (reliability first, nothing smart yet) → Tier 1 rules engine + calculator on the PS's own numbers → WhatsApp Flow for structured intake → Gemini wired in for Tier 2 + generation, caching from day one → Tier 3 prudential router with real rule logic + mocked per-branch data, clearly commented as mocked → language-select buttons. Two generically sound habits worth keeping regardless of exact timing: deploy to a real host early rather than demoing off `localhost`, and stop writing new code with real buffer time left before any deadline — that time is worth more spent rehearsing the demo and fixing what breaks live than adding one more feature.

Phase 1 — Internal pitch: beneficiary story first, not the tech. Live WhatsApp demo. Show the transparent "why" trail. State plainly what's mocked and why it's mocked (the honest framing from §6.1's table is genuinely a Problem Understanding asset, not just a caveat). Every teammate ready on every component.

Phase 2 — Gap before nationals: OpenRouter fallback cascade + IndicTrans2 translation bridge. Voice notes via AI4Bharat ASR/TTS. Wider partner data coverage. Pressure-test against deliberately weird inputs.

Phase 3 — Finale: same core story, deeper polish, off the dev tunnel and onto a real deployment (EC2, already proven via SHIELD). Web dashboard if there's time. Close on scalability beyond NSFDC — NBCFDC runs a near-identical channel-finance structure for OBCs.

12. Open risks / verify before you're committed

Eligibility numbers (§6.1) — genuinely unresolved across sources, including NSFDC's own pages. A fourth research document has since surfaced a *fifth* distinct set of numbers for the same schemes, disagreeing with the other four *and* with my own direct nsfdc.nic.in fetch. That's now a firm pattern, not bad luck — stop feeding this question more AI research and have someone call an SCA office or email the PS's nodal ministry contact directly. Default to the PS's own stated figures until then.

Twilio Sandbox vs. Cloud API — confirm which you're actually running on; Sandbox session behavior is a leading suspect for GrantBot's original reliability issues.

MUJ's internal-round nomination quota — confirmed 45 primary + 5 waitlisted teams, per direct prior experience clearing MUJ's internal round before (worth a quick reconfirm at Monday's SPOC meeting, since this can shift year to year, but it's real lived experience, not a guess). Two things are confirmed nationally regardless of institution: 6 members per team with at least 1 female, and a max of 2 problem statements per team.

MUJ's specific submission requirements — at least one other college's SIH page explicitly required human-only demo narration with no AI generation; confirm MUJ's own rule.

Open-source fallback models' actual Hindi output quality — untested for this use case; that's exactly why the translator bridge (§7) exists rather than trusting it directly.

The richer scheme universe (Udyam Nidhi, VCF-SC, Stand-Up India, PM-USP CSIS) is real and could be a scope-expansion story for the "Impact & Scalability" pitch — but the PS names three specific products; get those fully

right before reaching for more.